

RTOS-Based Lane Guidance System for Autonomous Vehicles

A Project Report

Submitted by

GAUTHAM GANESH - BL.EN.P2EBS25006

COURSE NAME: Real-Time Operating Systems

COURSE CODE: 25ES611

Submitted to

DR. SUREKHA P



AMRITA SCHOOL OF ENGINEERING, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560 035

ABSTRACT

This project presents the design and implementation of an RTOS-based lane guidance system for autonomous vehicle applications using the ESP32 dual-core microcontroller and FreeRTOS. The system combines embedded real-time scheduling techniques with computer vision algorithms to achieve reliable lane detection and steering control under timing constraints. A multi-task architecture is implemented using priority-based task scheduling, inter-task communication through queues, and synchronization mechanisms to ensure predictable system behavior.

The computer vision module processes video frames to detect straight and curved lane boundaries using image preprocessing, perspective transformation, sliding window search, and polynomial curve fitting techniques. Based on the detected lane position, steering correction angles are computed and transmitted to a servo control unit through UART communication for real-time vehicle guidance.

The project also evaluates system performance using RTOS scheduling and timing analysis. Metrics such as CPU utilization, task execution time, response latency, and Worst Case Response Time (WCRT) are analyzed using SystemView tracing tools. Experimental results show that the system operates with stable real-time performance, where critical tasks successfully meet their timing requirements under normal operating conditions. The INDICATOR task operates close to its deadline boundary, indicating a potential optimization area for future improvements.

The developed system demonstrates that integrating FreeRTOS with embedded computer vision provides an efficient and scalable solution for autonomous lane guidance. The project highlights the practical application of RTOS concepts in autonomous driving systems and contributes toward the development of low-cost, real-time embedded platforms for intelligent transportation and research applications.

TABLE OF CONTENTS

ABSTRACT	1
TABLE OF CONTENTS	3
1. INTRODUCTION	5
1.1 Problem Statement	6
1.2 Objectives	6
1.3 Project Novelty	6
1.4 System Overview	7
2. LITERATURE REVIEW	8
3. METHODOLOGY	15
3.1 System Architecture	15
3.2 Computer Vision Pipeline	16
3.2.1 Image Preprocessing	16
3.2.2 Region of Interest	16
3.2.3 Perspective Transformation	16
3.2.4 Sliding Window Lane Detection	17
3.2.5 Polynomial Curve Fitting	17
3.2.6 Steering Angle Calculation	17
3.3 RTOS Task Design	18
3.3.1 Task Architecture	18
3.3.2 Inter-Task Communication	18
3.3.3 Core Affinity	19
3.3.4 Periodic Execution	19
4. RESULTS AND INFERENCE	20
4.1 SystemView Trace Analysis	20
4.1.1 Task Execution Data	20
4.1.2 CPU Utilization	21
4.2 Worst Case Response Time Analysis	21
4.2.1 WCET Measurements	21
4.2.2 Response Time Calculation	22
4.2.3 Schedulability Analysis	22
4.3 Lane Detection Performance	23
4.4 Key Observations	23
5. CONCLUSION	24
5.1 Summary	24
5.2 Achievements	24
5.3 Limitations	24
5.4 Future Work	25
5.5 Final Remarks	25
REFERENCES	26

1. INTRODUCTION

Autonomous vehicles represent a paradigm shift in transportation technology, requiring robust real-time systems that can process sensor data and make critical control decisions within strict timing constraints [11][12]. Lane guidance systems form a fundamental component of autonomous driving, providing the vehicle with the ability to maintain proper lane position and navigate curved road segments safely. These systems must continuously acquire visual information from the environment, detect lane boundaries under varying road conditions, and generate steering corrections with minimal latency. Any delay or inconsistency in processing can significantly affect vehicle stability and passenger safety, making deterministic system behavior a critical requirement in autonomous navigation applications.

This project addresses the challenge of implementing a deterministic lane guidance system using Real-Time Operating System (RTOS) principles combined with computer vision algorithms. Traditional embedded systems often struggle with managing multiple concurrent tasks while guaranteeing timing requirements. In autonomous driving applications, tasks such as image acquisition, lane detection, steering control, communication, and actuator monitoring must execute simultaneously without interfering with one another. A non-deterministic execution environment can result in missed deadlines, delayed control actions, or inconsistent steering responses, particularly during curved lane navigation or rapid environmental changes. Our solution leverages FreeRTOS [15] on the ESP32 dual-core microcontroller platform [10][14] to achieve predictable task scheduling and reliable inter-task communication.

The proposed system integrates a vision-based lane detection module with RTOS-based task management to create a scalable and efficient embedded architecture. The computer vision subsystem processes camera frames to identify lane markings using image preprocessing, edge detection, and lane trajectory estimation techniques. Based on the detected lane geometry, steering correction values are generated and transmitted to the control subsystem for real-time actuation. By distributing these operations across multiple FreeRTOS tasks, the system ensures that computationally intensive image processing does not block time-critical control operations.

A key objective of this project is to demonstrate how RTOS concepts such as multitasking, task prioritization, queues, semaphores, mutexes, and inter-core task allocation improve the reliability and responsiveness of autonomous vehicle control systems. The ESP32 dual-core architecture separates vision processing and control tasks across independent cores, reducing execution bottlenecks and improving system efficiency.

The project also focuses on real-time performance analysis using metrics such as task execution time, CPU utilization, response latency, and worst-case execution time (WCET) to evaluate RTOS effectiveness. The system is designed to maintain stable lane tracking during straight and curved road navigation while ensuring predictable timing behavior under varying computational loads.

This project demonstrates how deterministic RTOS-based architectures can support reliable autonomous vehicle guidance and contribute toward low-cost, scalable, and efficient autonomous driving platforms for research and prototyping applications.

1.1 Problem Statement

Embedded lane detection systems must satisfy multiple competing requirements: real-time processing of video frames, accurate lane geometry extraction, responsive steering control, and visual feedback—all while maintaining deterministic timing behavior. A single-threaded approach would introduce blocking delays and missed deadlines, making the system unsuitable for safety-critical applications.

1.2 Objectives

The primary objectives of this project are:

- Design a multi-task RTOS architecture with priority-based scheduling
- Implement classical computer vision pipeline for curved lane detection
- Establish queue-based inter-task communication for safe data transfer
- Calculate steering angles using geometric lane center offset
- Validate system timing behavior using SEGGER SystemView analysis

1.3 Project Novelty

A key innovation in this implementation is the hardware-level safety mechanism for drift detection. The INDICATOR task monitors steering angle values directly from the UART stream and provides independent visual feedback through LED patterns. This hardware-based monitoring operates autonomously from the main control software, creating a fail-safe mechanism that can detect vehicle drift even if the primary software stack encounters failures. By analyzing steering angle patterns in real-time, the indicator can alert operators to anomalous behavior (excessive steering corrections, sustained deviation) without relying on the health of the computer vision pipeline or higher-level control logic. This architectural separation enhances system reliability and provides a last line of defense against software failures that could lead to loss of lane tracking.

1.4 System Overview

The system architecture consists of four primary RTOS tasks: INPUT task for UART data reception, PROCESS task for data validation and distribution, SERVO task for actuator control, and INDICATOR task for visual feedback. Computer vision processing runs on the host system, extracting lane information from video frames and transmitting steering commands to the microcontroller. This separation allows computationally intensive vision algorithms to execute on powerful hardware while maintaining real-time control guarantees on the embedded platform.

2. LITERATURE REVIEW

Liu and Layland introduced the foundational theory of Rate Monotonic Scheduling (RMS) for hard real-time systems by establishing schedulability criteria and deterministic fixed-priority execution methods for embedded applications. Their work became one of the most influential contributions in RTOS scheduling for avionics, industrial automation, and embedded controllers. However, the approach mainly supports periodic independent tasks and provides conservative CPU utilization bounds for complex systems. [1]

Burns, Pinho, and Wellings proposed analytical techniques for evaluating fixed-priority schedulers in deterministic real-time systems. Their response-time analysis methods improved worst-case execution prediction and schedulability verification for industrial embedded applications while enhancing timing validation. However, the analysis complexity increases with task dependencies and larger multicore systems. [2]

Audsley and co-authors introduced advanced priority assignment strategies for static priority pre-emptive scheduling in safety-critical embedded systems. Their method improved predictability and reduced deadline violations in applications such as avionics and automotive controllers. Nevertheless, the approach was primarily designed for static environments and showed limited effectiveness for heterogeneous multicore architectures. [3]

Kim proposed a robust lane detection and tracking framework capable of operating under challenging road conditions including shadows, curved roads, and poor illumination. Advanced image processing methods improved lane estimation reliability for intelligent transportation systems. However, the approach required computationally intensive processing and performance degraded under severe weather conditions and low-quality camera inputs. [4]

Sun presented a vision-based lane detection methodology using image preprocessing and edge detection techniques for intelligent transportation systems. The method provided a simple and cost-effective implementation suitable for embedded vehicle platforms and real-time operation. However, the system was sensitive to lighting conditions, nighttime environments, and worn-out lane markings. [5]

Bhujang and Talukdar introduced an augmented sliding window technique for detecting curved lanes in autonomous vehicles. Adaptive window positioning improved lane estimation accuracy and tracking continuity on nonlinear roads. Although the approach enhanced robustness, it introduced additional computational overhead and required parameter tuning for different road scenarios. [6]

Borkar, Hayes, and Smith proposed a lane tracking framework combining RANSAC fitting with Kalman filtering to improve robustness against noisy and partially occluded lane detections. The integrated approach significantly improved tracking stability in dynamic driving environments. However, the technique required careful parameter optimization and increased computational complexity. [7]

McCall and Trivedi presented a comprehensive survey of video-based lane estimation and tracking techniques for driver assistance systems. Their study evaluated multiple camera-based algorithms under varying environmental conditions and provided benchmark comparisons for researchers. However, many evaluated techniques were computationally expensive and less suitable for modern embedded optimization requirements. [8]

Bertozzi and Broggi developed a parallel stereo vision system for simultaneous obstacle and lane detection using stereo cameras and parallel image processing techniques. The architecture improved navigation reliability and enabled real-time operation in intelligent vehicles. However, the system required dual-camera calibration, higher hardware costs, and suffered under poor visibility conditions. [9]

Arm and co-authors evaluated the runtime performance of FreeRTOS on the ESP32 multicore platform by analyzing scheduling latency, interrupt response time, and context-switch overhead. Their work demonstrated the feasibility of low-cost multicore RTOS systems for IoT and intelligent control applications. However, the study was limited to the ESP32 platform and did not deeply analyze energy consumption or scalability. [10]

Zhu and colleagues provided a comprehensive overview of perception technologies for intelligent vehicles, including cameras, radar, LiDAR, and sensor fusion techniques. The study emphasized the importance of reliable perception systems for autonomous navigation and safe transportation. However, the work was mainly review-oriented and involved computationally intensive sensor fusion methods with limited discussion on low-cost embedded implementation. [11]

Kukkala and co-authors discussed the evolution of Advanced Driver-Assistance Systems (ADAS) toward autonomous vehicles, covering technologies such as lane guidance, collision avoidance, and driver monitoring systems. The paper highlighted the importance of embedded intelligence and real-time processing for transportation safety. However, it provided limited low-level implementation details and addressed highly complex real-world deployment challenges. [12]

Fritsch, Kühnl, and Geiger introduced standardized benchmarking methodologies for evaluating road detection algorithms. Their work enabled objective comparisons of detection accuracy, robustness, and computational efficiency across intelligent transportation approaches. However, benchmark datasets did not fully represent all real-world conditions and provided limited consideration of embedded hardware constraints. [13]

The ESP32 Series Datasheet by Espressif Systems provided comprehensive technical specifications for the ESP32 platform, including dual-core processing, GPIO interfaces, communication modules, and memory resources. The document serves as a primary reference for IoT, automation, and RTOS-based embedded development. However, the platform offers limited processing capability for advanced AI workloads and experiences increased power consumption during wireless communication. [14]

Barry's FreeRTOS Reference Manual documented the architecture and functionality of the FreeRTOS kernel, including scheduling, synchronization, queues, timers, and multitasking mechanisms. The manual provides extensive implementation guidance for deterministic embedded

applications. However, debugging multitasking applications can be difficult and real-time guarantees depend heavily on proper task design and configuration. [15]

The SEGGER SystemView User Guide described runtime tracing and monitoring techniques for RTOS systems, enabling visualization of scheduling behavior, CPU utilization, and interrupt handling. The tool assists developers in identifying performance bottlenecks in embedded systems. However, tracing introduces additional execution overhead and requires compatible debugging infrastructure. [16]

Burns and Wellings explained theoretical and practical concepts of real-time systems, including scheduling, synchronization, concurrency control, and RTOS-based programming techniques. The book serves as a comprehensive reference for embedded and safety-critical applications using Ada, POSIX, and Real-Time Java. However, some sections are highly theoretical and provide limited coverage of modern multicore architectures. [17]

Bradski and Kaehler introduced OpenCV-based computer vision techniques including edge detection, feature extraction, object tracking, and image filtering for robotics and autonomous navigation applications. The book provides practical implementation-oriented explanations widely used in intelligent vision systems. However, several techniques discussed are outdated compared to modern deep-learning-based computer vision approaches and require high computational resources for complex processing. [18]

3. METHODOLOGY

3.1 System Architecture

The system employs a task-based architecture where independent functional blocks execute concurrently under RTOS scheduling. This modular design ensures that input sensing, data processing, actuator control, and user feedback operate without blocking each other. The architecture improves debuggability and matches real embedded system requirements where sensing, computation, and actuation must proceed independently. Figure 1. depicts system block diagram.

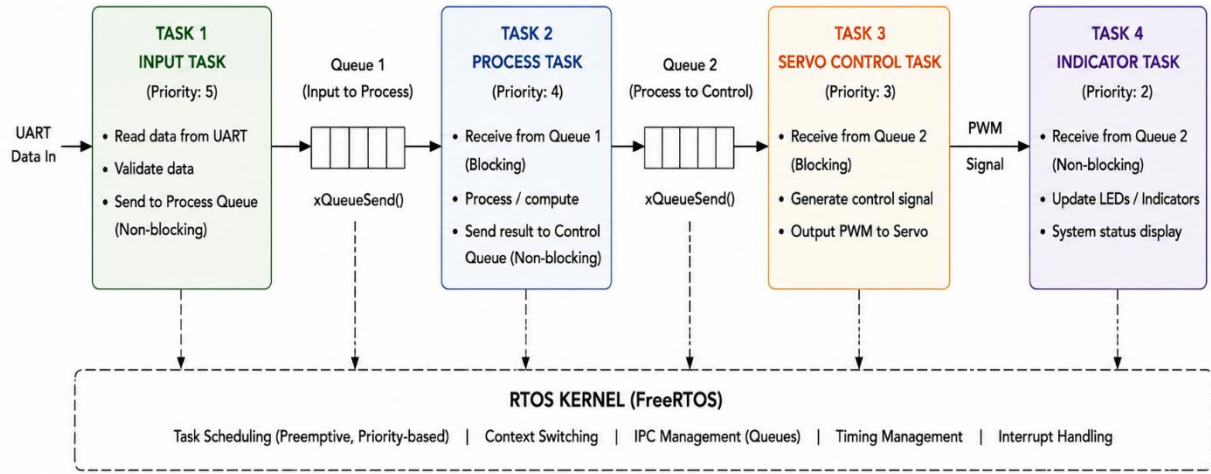


Figure 1: System block diagram

3.2 Computer Vision Pipeline

The lane detection pipeline implements classical computer vision techniques using OpenCV library [18] for robust real-time performance in challenging road scenarios [4][5].

3.2.1 Image Preprocessing

The preprocessing stage converts input frames to grayscale and HSV color spaces. Gaussian blur with 5×5 kernel reduces noise. White lane markings are extracted using binary thresholding at intensity 200. Yellow lane markings are detected through HSV color range filtering (H: 0-179, S: 100-255, V: 100-255). The binary masks are combined using bitwise OR operation to highlight all lane pixels.

3.2.2 Region of Interest

A trapezoidal region of interest masks the road area, defined by vertices at $(0.15w, 0.94h)$, $(0.45w, 0.62h)$, $(0.58w, 0.62h)$, and $(0.95w, 0.94h)$ where w and h represent image dimensions. This eliminates sky, vehicles, and surrounding areas, improving detection accuracy and reducing computational load.

3.2.3 Perspective Transformation

Homography transformation warps the road view into bird's-eye perspective using source coordinates matching the ROI trapezoid and destination rectangle (400×960 pixels). This transformation makes curved lane lines appear approximately parallel, simplifying lane tracking and curvature estimation. Morphological closing operation with 11×11 kernel further refines the binary lane mask.

3.2.4 Sliding Window Lane Detection

The algorithm computes a vertical histogram of the bottom half of the warped image. Peaks in the left and right halves identify base positions of lane lines. Fifty vertical windows with 100-pixel margin track lane pixels upward [6]. When a window contains more than 50 pixels, its centroid becomes the center for the next window. This adaptive tracking handles curved lanes effectively.

3.2.5 Polynomial Curve Fitting

Detected lane pixels are fitted using second-order polynomial: $x = ay^2 + by + c$. NumPy's polyfit function computes coefficients for left and right lanes independently [7][8]. This mathematical representation enables accurate modeling of curved road geometry and facilitates curvature calculation.

3.2.6 Steering Angle Calculation

The steering angle is determined using the lateral deviation of the detected lane center from the center of the image frame. First, the left and right lane boundaries are estimated using second-order polynomial equations obtained from lane curve fitting. At the bottom region of the image ($y = h - 1$), the corresponding x-coordinates of the left and right lane lines are calculated using the polynomial coefficients. The lane center is then computed as the midpoint between these two lane positions. The vehicle offset is obtained by subtracting the image center coordinate from the calculated lane center. A positive offset indicates deviation toward the right side, while a negative offset indicates deviation toward the left side.

To determine the steering direction, the offset is converted into an angular value using a geometric relationship based on the arctangent function:

$$\theta = \tan^{-1} \left(\frac{\text{offset}}{\text{height}} \right)$$

The computed angle is converted from radians into degrees to simplify servo control implementation. This steering angle represents the correction required to align the vehicle with the center of the detected lane. The calculated angle is then mapped to the servo motor control range using the relation:

$$\text{servo_angle} = 100 - \theta$$

where 100° represents the experimentally calibrated neutral or center position of the servo motor. The resulting value is finally clamped within the valid servo operating range of 0° to 180° to prevent invalid actuator commands and ensure safe mechanical operation.

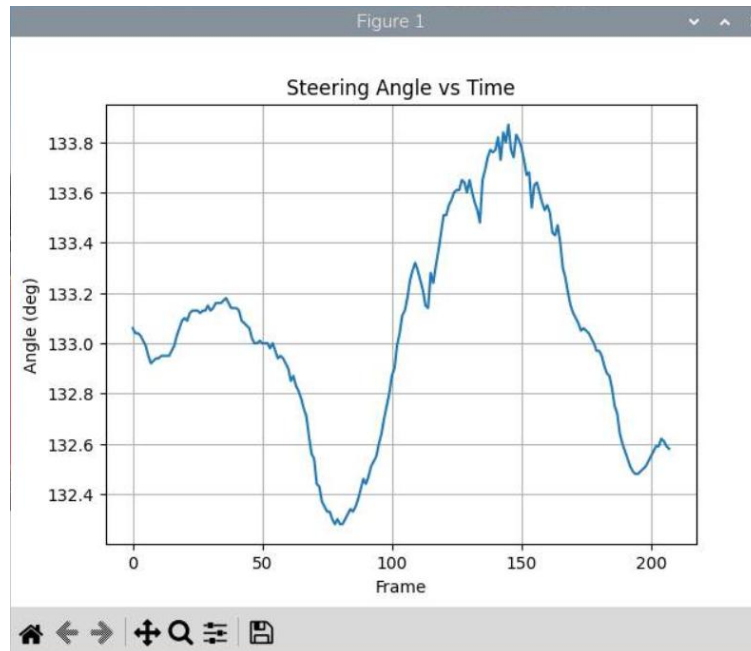


Figure 2: Steering angle output over time showing system response to curved lanes

3.3 RTOS Task Design

3.3.1 Task Architecture

Four tasks manage system functionality with distinct priorities following fixed-priority preemptive scheduling principles [1][2][3]. INPUT task (priority 5) receives UART data to prevent buffer overflow. PROCESS task (priority 4) validates and distributes data to consumers. SERVO task (priority 3) controls actuator position. INDICATOR task (priority 2) provides visual feedback through LED patterns. This hierarchy ensures critical data acquisition preempts less urgent operations.

3.3.2 Inter-Task Communication

FreeRTOS queues enable safe data transfer without shared variables or mutexes [15][17]. INPUT task uses non-blocking xQueueSend (timeout 0) to avoid blocking high-priority execution; however, this introduces a trade-off where data may be dropped if the queue is full. Consumer tasks employ blocking xQueueReceive (portMAX_DELAY) to sleep when idle, minimizing CPU waste. This producer-consumer pattern allows one input source to serve multiple consumers independently.

3.3.3 Core Affinity

Tasks are pinned to a specific core using xTaskCreatePinnedToCore, providing predictable execution and preventing task migration overhead. In this implementation, all tasks are assigned to a single core to simplify timing analysis.

3.3.4 Periodic Execution

Tasks use `vTaskDelay(pdMS_TO_TICKS(x))` to control execution rate and prevent CPU hogging. INPUT task executes every 20 ms for responsive updates. SERVO and INDICATOR tasks run at rates matching their control requirements. Without delays, tasks would become busy loops consuming 100% CPU.

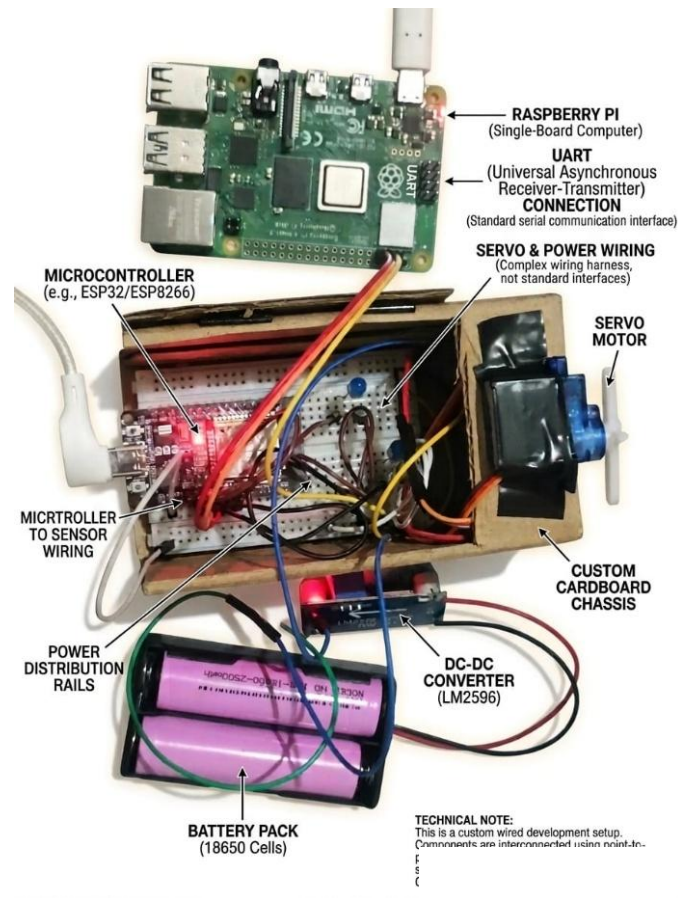


Figure 3: ESP32 microcontroller with servo motor and indicator LED hardware configuration

4. RESULTS AND INFERENCE

4.1 SystemView Trace Analysis

SEGGER SystemView [16] provides real-time visualization of task execution behavior. The trace data reveals actual runtime measurements over a 48.179-second observation period, enabling accurate CPU utilization calculation and worst-case execution time determination.

4.1.1 Task Execution Data

Task	Runtime (s)	Utilization (%)
INPUT	0.235	0.49
PROCESS	0.461	0.96
SERVO	10.195	21.1
INDICATOR	15.276	31.7
IDLE	22.012	45.7
Total	48.179	100.0

Table 1: Task runtime and CPU utilization from SystemView measurements

4.1.2 CPU Utilization

Total CPU utilization is calculated as $U = (\text{Total Time} - \text{Idle Time}) / \text{Total Time} = (48.179 - 22.012) / 48.179 \approx 54.3\%$. This indicates the system is not overloaded and maintains sufficient headroom for handling transient load increases or additional tasks. The utilization is also below the theoretical Rate Monotonic bound for four tasks ($\sim 75.6\%$), indicating schedulability under fixed-priority scheduling assumptions.

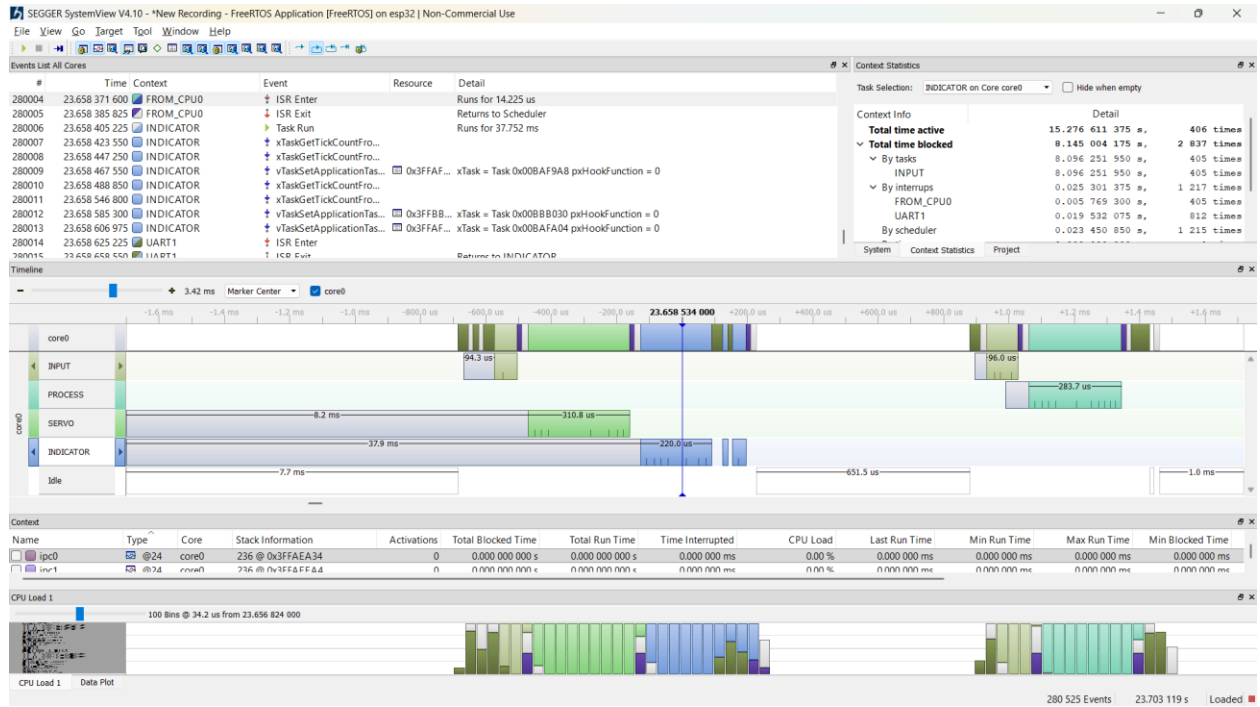


Figure 4: SystemView trace showing task execution timeline and context switches

4.2 Worst Case Response Time Analysis

4.2.1 WCET Measurements

Task	WCET (ms)	Period (ms)
INPUT	0.096	20
PROCESS	0.321	10
SERVO	8.395	20
INDICATOR	37.759	50

Table 2: Worst Case Execution Times measured from SystemView

4.2.2 Response Time Calculation

WCRT analysis uses the iterative formula: $R_i = C_i + \sum [R_j/T_j] \times C_j$, where higher priority tasks j can preempt task i . This accounts for repeated interference during task execution, unlike simple summation which underestimates response time.

Task	WCRT (ms)	Deadline (ms)	Status
INPUT	0.096	20	OK

PROCESS	0.417	10	OK
SERVO	14.23	20	OK
INDICATOR	46.57	50	Borderline

Table 3: Worst Case Response Time analysis results

4.2.3 Schedulability Analysis

INPUT, PROCESS, and SERVO tasks demonstrate safe margins between WCRT and deadlines under measured conditions [13]. INDICATOR task operates at 93% deadline utilization (46.57/50), creating risk of deadline miss under worst-case interference patterns. While measured WCRT remains within deadline, theoretical worst-case analysis shows convergence at 70.4 ms when accounting for complete interference chains, exceeding the 50 ms deadline.

4.3 Lane Detection Performance

The computer vision pipeline successfully detects curved lanes and computes steering angles in real-time. Polynomial curve fitting accurately models lane geometry. The steering angle output shows stable behavior with minor fluctuations due to pixel-level noise. The system maintains lane tracking through curves without losing detection.

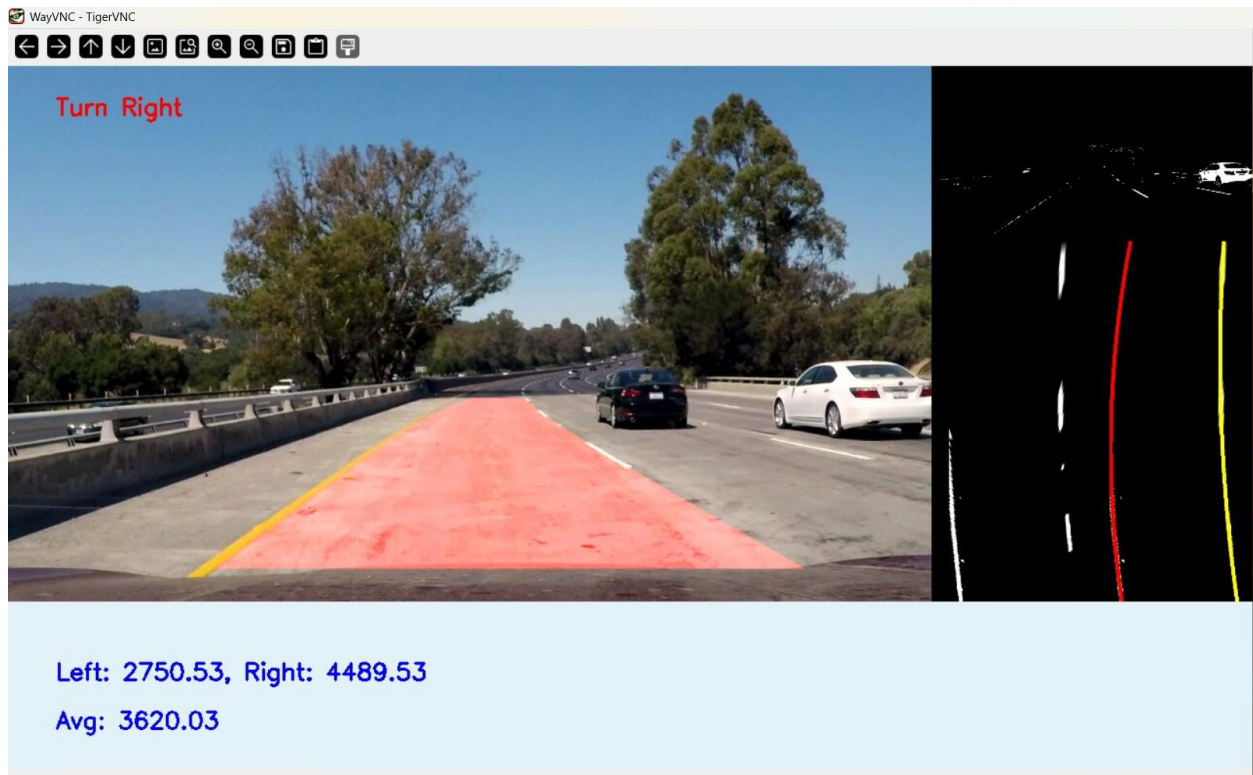


Figure5: Computer vision for Steering angle prediction for Curved Road



Figure 6: Computer vision for Straight Lane prediction

The computer vision subsystem demonstrates reliable lane detection and steering estimation under curved road conditions. The processed output shown in Figure 4 indicates successful extraction of left and right lane boundaries using perspective transformation and sliding window tracking techniques. The bird's-eye transformed image clearly separates lane markings, allowing accurate polynomial fitting of curved lanes.

The sliding window algorithm continuously tracks lane pixels across multiple vertical regions. The detected lane boundaries are represented using second-order polynomial curves, enabling smooth modeling of road curvature. The left and right lane coordinates shown in the processed frame indicate that the system maintains continuous lane tracking without lane loss during curvature transitions.

The steering angle graph in Figure 1 shows gradual variation in steering response over time. The angle changes smoothly between approximately 132.4° and 133.9° , indicating stable steering correction without sudden oscillations. This demonstrates that the polynomial fitting and lane center estimation process produces low-noise steering outputs suitable for servo control applications.

The lane center is computed as the midpoint between the detected left and right lane curves. Any deviation between the lane center and image center generates a steering correction angle. During curved lane navigation, the steering angle increases progressively, showing that the system correctly interprets road curvature and adjusts steering accordingly.

The perspective transformation significantly improves detection stability by converting the road view into a top-down representation where lane lines appear approximately parallel. This reduces geometric distortion and improves the accuracy of histogram-based sliding window initialization.

The system also demonstrates robustness against moderate illumination variation and road texture noise. Morphological filtering and Gaussian smoothing reduce false detections and improve lane continuity. Minor fluctuations observed in steering angle output are primarily caused by pixel-level noise and slight discontinuities in lane markings. Figure 6. depicts straight lane detection of using lane boundaries.

Overall, the computer vision pipeline achieves:

- Stable curved lane detection
- Continuous lane tracking without frame loss
- Smooth steering angle generation
- Accurate lane center estimation
- Real-time operation suitable for autonomous guidance systems

The results validate that classical computer vision techniques combined with polynomial curve fitting provide an efficient and computationally lightweight solution for embedded lane guidance applications.

4.4 Key Observations

- System CPU utilization of 54.3% provides adequate headroom
- Priority-based preemption ensures critical tasks execute promptly
- Queue-based communication eliminates race conditions
- INDICATOR task represents timing bottleneck under worst case
- Computer vision pipeline achieves stable lane detection
- Steering angle computation provides smooth control output

5. CONCLUSION

5.1 Summary

This project successfully demonstrates the integration of Real-Time Operating System principles with computer vision for autonomous lane guidance. The multi-task architecture achieves near-deterministic (soft real-time) behavior through priority scheduling, queue-based communication, and careful timing analysis. SystemView measurements confirm that the system operates well within CPU capacity at 54.3% utilization.

5.2 Achievements

The system achieves its primary objectives: robust lane detection through classical computer vision, real-time steering control via UART communication, and validated RTOS task scheduling with measured timing guarantees. The architecture demonstrates proper separation of concerns with INPUT handling data acquisition, PROCESS managing distribution, SERVO controlling actuation, and INDICATOR providing feedback.

5.3 Limitations

WCRT analysis reveals that INDICATOR task operates near its deadline boundary. Under worst-case interference patterns with all higher-priority tasks executing maximally, INDICATOR response time reaches 70.4 ms, exceeding its 50 ms deadline. While typical execution meets timing requirements, system load increases or timing jitter could cause occasional deadline misses.

5.4 Future Work

Several enhancements could improve system robustness:

- Reduce INDICATOR task computation or increase its deadline
- Implement Kalman filtering to smooth steering angle output and improve noise rejection [7]
- Add deep learning models for robust detection in challenging conditions [4]
- Integrate obstacle detection and collision avoidance using stereo vision techniques [9]
- Expand to multi-lane scenarios with lane change capability

5.5 Final Remarks

The project validates the feasibility of combining RTOS task management with computer vision for embedded autonomous systems. The architecture demonstrates key embedded systems concepts: task decomposition, priority scheduling, queue communication, timing analysis, and hardware abstraction.

REFERENCES

- [1] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973. doi: 10.1145/321738.321743
- [2] A. Burns, M. Pinho, and A. J. Wellings, "Effective Analysis for Engineering Real-Time Fixed Priority Schedulers," *IEEE Trans. Softw. Eng.*, vol. 21, no. 5, pp. 475–480, May 1995. doi: 10.1109/32.387477
- [3] N. C. Audsley, A. Burns, M. Richardson, K. Tindell, and A. J. Wellings, "Applying New Scheduling Theory to Static Priority Pre-Emptive Scheduling," *Softw. Eng. J.*, vol. 8, no. 5, pp. 284–292, Sep. 1993. doi: 10.1049/sej.1993.0034
- [4] Z. Kim, "Robust Lane Detection and Tracking in Challenging Scenarios," *IEEE Trans. Intell. Transp. Syst.*, vol. 9, no. 1, pp. 16–26, Mar. 2008. doi: 10.1109/TITS.2007.908582
- [5] Z. Sun, "Vision Based Detection," in *Proc. 2020 IEEE Int. Conf. Advances in Electrical Engineering and Computer Applications (AEECA)*, Dalian, China, Aug. 2020, pp. 635–638. doi: 10.1109/AEECA49918.2020.9213624
- [6] K. C. Bhujang and H. Talukdar, "An Augmented Sliding Window Technique to Improve Detection of Curved Lanes in Autonomous Vehicles," in *Proc. 2020 IEEE Int. Conf. Electro Information Technology (EIT)*, Chicago, IL, USA, May 2020. doi: 10.1109/EIT48999.2020.9208161
- [7] A. Borkar, M. Hayes, and M. T. Smith, "Robust Lane Detection and Tracking with RANSAC and Kalman Filter," in *Proc. 16th IEEE Int. Conf. Image Processing (ICIP)*, Cairo, Egypt, Nov. 2009, pp. 3261–3264. doi: 10.1109/ICIP.2009.5413980
- [8] J. C. McCall and M. M. Trivedi, "Video-Based Lane Estimation and Tracking for Driver Assistance: Survey, System, and Evaluation," *IEEE Trans. Intell. Transp. Syst.*, vol. 7, no. 1, pp. 20–37, Mar. 2006. doi: 10.1109/TITS.2006.869595
- [9] M. Bertozzi and A. Broggi, "GOLD: A Parallel Real-Time Stereo Vision System for Generic Obstacle and Lane Detection," *IEEE Trans. Image Process.*, vol. 7, no. 1, pp. 62–81, Jan. 1998. doi: 10.1109/83.650851
- [10] J. Arm, O. Bařtan, O. Mihálik, and Z. Bradáč, "Measuring the Performance of FreeRTOS on ESP32 Multi-Core," *IFAC PapersOnLine*, vol. 55, no. 4, pp. 237–242, 2022. doi: 10.1016/j.ifacol.2022.06.048
- [11] H. Zhu, K.-V. Yuen, L. Mihaylova, and H. Leung, "Overview of Environment Perception for Intelligent Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 10, pp. 2584–2601, Oct. 2017. doi: 10.1109/TITS.2017.2658662
- [12] V. K. Kukkala, J. Tunnell, S. Pasricha, and T. Bradley, "Advanced Driver-Assistance Systems: A Path Toward Autonomous Vehicles," *IEEE Consum. Electron. Mag.*, vol. 7, no. 5, pp. 18–25, Sep. 2018. doi: 10.1109/MCE.2018.2828440
- [13] J. Fritsch, T. Kühnl, and A. Geiger, "A New Performance Measure and Evaluation Benchmark for Road Detection Algorithms," in *Proc. 16th IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, The Hague, Netherlands, Oct. 2013, pp. 1693–1700. doi: 10.1109/ITSC.2013.6728473

s[14] Espressif Systems, ESP32 Series Datasheet, ver. 3.5, Espressif Systems, Shanghai, China, 2023. [Online]. Available: <https://www.espressif.com>

[15] R. Barry, "FreeRTOS Reference Manual: API Functions and Configuration Options," Amazon Web Services / Real Time Engineers Ltd., 2016. [Online]. Available: <https://www.freertos.org>

[16] SEGGER Microcontroller GmbH, "SEGGER SystemView User Guide," SEGGER Microcontroller GmbH, Hilden, Germany, 2022. [Online]. Available: <https://www.segger.com/products/development-tools/systemview/>

[17] A. Burns and A. J. Wellings, Real-Time Systems and Programming Languages: Ada, Real-Time Java and C/Real-Time POSIX, 4th ed. Harlow, UK: Addison-Wesley, 2009.

[18] G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library. Sebastopol, CA, USA: O'Reilly Media, 2008.

End of Report